

Project Overview

Architecture · Technology · Data Flow

A prototype electronic voting system that combines **quantum cryptography**, **post-quantum encryption**, and **blockchain immutability** to ensure votes are private, tamper-proof, and verifiable.

File Structure

```
IPD-Voting-1st-Prototype/
■
■■■ vol1_encoding.py # Module 1 – Quantum vote encoding (Qiskit)
■■■ vol2_bb84.py # Module 2 – BB84 Quantum Key Distribution (Qiskit)
■■■ vol3_pqc.py # Module 3 – Post-Quantum Encryption (Fernet/SHA-256)
■■■ vol4_blockchain.py # Module 4 – Blockchain storage & validation
■
■■■ demo1.py # Full end-to-end integrated demo (all 4 modules)
■
■■■ test_vol1_encoding.py # Unit tests for Module 1
■■■ test_vol2_bb84.py # Unit tests for Module 2
■■■ test_vol3_pqc.py # Unit tests for Module 3
■■■ test_vol4_blockchain.py # Unit tests for Module 4 (10 tests, all passing)
```

Each `vol{N}` module is independently runnable (`python vol1_encoding.py`) and independently testable (`python test vol{N} *.py`). They connect together only in `demo1.py`.

Architecture

```

■ VOTER ■
■ selects a candidate from the list ■
■
▼
■ MODULE 1 – Quantum Vote Encoding ■
■ vol1_encoding.py ■
■ ■
■ • Maps candidate → binary index (e.g. "Diya" → "01") ■
■ • Encodes that index into qubits using X gates ■
■ • Simulates measurement on a quantum circuit ■
■ • Decodes measured bits back → confirms correct candidate ■
■ ■
■ Tech: Qiskit QuantumCircuit + AerSimulator ■
■ confirmed candidate name

```

[illegible]

```

■ demo1.py ■
■ ■
■ • Retrieve encrypted payload from the chain block ■
■ • Convert hex back to bytes ■
■ • Decrypt using the same BB84-derived key ■
■ • Compare decrypted result to original vote → must match ■

```

Technology Stack

Layer	Technology	Why
Quantum simulation	[Qiskit](https://qiskit.org/) + <code>qiskit-aer</code>	Industry-standard quantum computing framework; AerSimulator runs quantum circuits locally without real hardware
Quantum key exchange	BB84 protocol (custom impl.)	Provably secure by the laws of quantum mechanics — eavesdropping is physically detectable
Symmetric encryption	<code>cryptography.fernet</code> (AES-128 in CBC+HMAC)	Authenticated encryption — ciphertext includes an integrity tag so tampering is caught at decryption
Key derivation	<code>hashlib.sha256</code>	Stretches a short BB84 bit-string into a full 256-bit key deterministically
Blockchain	Pure Python (<code>hashlib</code> , <code>json</code>)	No external chain library needed for prototype; transparent, auditable, zero dependencies
Language	Python 3.12	Ecosystem support for all above libraries
Testing	Plain Python <code>assert</code> + print-based tests	Lightweight, no pytest dependency — matches the project's self-contained approach

Module-by-Module Breakdown

`vol1\_encoding.py` — Quantum Vote Encoding

What it does:

Maps a candidate selection to a binary index, encodes it into quantum bits using X (NOT) gates, then measures the circuit to confirm the encoding is correct.

Why it exists:

This represents the "quantum" nature of the vote itself. In a real system, the qubit state would be transmitted over a quantum channel. In this prototype, the simulation confirms the encoding/decoding

roundtrip is lossless.

Key function:

```
encode_vote(candidates: list, chosen_candidate: str) -> dict
# Returns: encoded_binary, measured_bits, decoded_candidate, circuit
```

`vol2\_bb84.py` — BB84 Quantum Key Distribution

What it does:

Simulates the BB84 protocol — the sender encodes random bits into qubits using random bases; the receiver measures with her own random bases; they keep only the bits where their bases happened to match.

Why it exists:

BB84 is the foundation of quantum cryptography. The key it produces cannot be intercepted without disturbing the quantum states, making any eavesdropping detectable. This key is then used to encrypt the vote.

Key functions:

```
generate_bb84_key(key_length: int) -> dict # one round of BB84
get_secure_key(min_length: int) -> list # loops until key is long enough
```

`vol3\_pqc.py` — Post-Quantum Encryption

What it does:

Takes the BB84 bit-list, hashes it with SHA-256 to produce a 256-bit AES key, then encrypts the vote candidate string using Fernet (AES-CBC + HMAC-SHA256).

Why it exists:

Raw BB84 output is too short and not in the right format for AES. SHA-256 key derivation solves both problems. Fernet adds authenticated encryption — decryption fails loudly if the wrong key is used or if the ciphertext was tampered with.

Key functions:

```
encrypt_vote(vote_data: str, bb84_key: list) -> bytes
decrypt_vote(ciphertext: bytes, bb84_key: list) -> str
```

`vol4\_blockchain.py` — Blockchain Storage

What it does:

Implements a linked list of immutable blocks. Each block stores one vote, timestamps itself, and hashes its entire contents + a pointer to the previous block's hash. Any alteration to any block breaks every subsequent hash.

Why it exists:

Blockchain provides **immutable audit trail** — no administrator, no attacker, not even the system itself can silently alter a recorded vote. The entire chain can be published publicly for independent verification.

Key classes and methods:

```
class Block:
    _compute_hash() # SHA-256 of (index + vote_data + prev_hash + timestamp)

class Blockchain:
    add_vote() # append a new block
    is_chain_valid() # walk chain, verify all hashes + links
    has_voter_voted() # double-vote prevention
    view_chain() # human-readable chain printout
    export_chain() # list[dict] for JSON serialisation
```

`demo1.py` — Integrated End-to-End Demo

What it does:

Runs all four modules in sequence for a single vote, printing the output of every step.

Flow:

```
1. encode_vote("Diya", candidates) # vol1
2. get_secure_key(min_length=8) # vol2
3. encrypt_vote(candidate, shared_key) # vol3 → ciphertext bytes
4. bc.add_vote(voter_id, ciphertext.hex()) # vol4 → stored on chain
5. bytes.fromhex(bc.chain[1].vote_data["encrypted_vote"])
   decrypt_vote(retrieved_bytes, shared_key) # vol3 → "Diya"
6. assert decrypted == original # integrity confirmed
```

Data Flow (Concrete Example)

```
Input: voter_id = "V001", chosen = "Diya"

vol1 → binary index : "01"
measured bits: "01"
decoded : "Diya" ✓ encoding correct

vol2 → BB84 shared key : [1, 1, 1, 0, 1, 1, 1, 1] ✓ keys match

vol3 → SHA-256("11101111") → 256-bit Fernet key
Fernet.encrypt("Diya") → ciphertext bytes
hex(ciphertext) → "6741414141..." ✓ encrypted

vol4 → Block #1 {
  voter_id : "V001"
  encrypted_vote: "6741414141..."
  timestamp : "2026-08-04T17:59:44Z"
  prev_hash : "1970e58b..." ← genesis hash
  hash : "0ac96e76..." ← SHA-256 of this block
}
is_chain_valid() → True ✓ on chain

verify → hex → bytes → Fernet.decrypt → "Diya"
```

```
"Diya" == "Diya" → True ✓ end-to-end complete
```

Why This Order of Development

Order	Module	Reason
1st	Blockchain	Needs no external dependencies; testable immediately with dummy data
2nd	Quantum encoding	Self-contained; only needs Qiskit
3rd	BB84 QKD	Depends on Qiskit; independent of encryption
4th	PQC encryption	Depends on BB84 key as input; testable with a hardcoded key
5th	Integration (demo1.py)	Connects well-tested modules; bugs are easy to isolate
6th	Streamlit UI	Built last, after the entire backend is verified

Building each module independently means: when something breaks in integration, you already know each piece works — so the bug must be in the connections, not the logic.

What's Coming Next

Module	Status	What it adds
Quantum eavesdropping detection	Planned	Detects if BB84 channel was compromised; aborts vote if so
True ML-KEM / ML-DSA (PQC)	Planned	Replaces Fernet with NIST-standardised post-quantum algorithms
Multi-voter support	Planned	Loop over multiple voters; tally at the end
Streamlit UI	Planned	Visual dashboard: cast vote, view chain, see tally